



AWS Solutions Architect Associate Cheat Sheets

These cheat sheets are provided for personal and non-commercial use only.

Please do not redistribute or upload these cheatsheets elsewhere.

Good luck on your exam!



S3 CheatSheet

Amazon S3 Overview

- **Object Storage:** Manages data as objects (files).
 - Key: Name of the object.
 - Value: Data itself (sequence of bytes).
 - Metadata: Additional info attached to the object.
- **Buckets:** Containers for objects, can have folders.
- **Unlimited Storage:** No need to manage infrastructure.

S3 Use Cases

- **General Purpose:** Content distribution, big data analytics, backups.
- **Static Website Hosting:** Host websites directly from S3.
- **Data Archiving:** Long-term storage using Glacier and Deep Archive.

S3 Bucket Configuration

- **Naming Rules:** Unique names, 3-63 characters, only lowercase letters, numbers, dots, and hyphens.
- **Restrictions:** Default 100 buckets, can request up to 1000.
- **Versioning:** Keeps multiple versions of an object.
- **Encryption:** Server-side (SSE-S3, SSE-KMS) and client-side encryption.



S3 CheatSheet

S3 Object Features

- **ETags:** Hash values to detect changes.
- **Checksums:** Ensures data integrity.
- **Metadata:** System-defined and user-defined metadata for organizing data.
- **Object Lock:** Prevents deletion or modification of objects.

S3 Storage Classes

- **Standard:** High durability and availability, frequent access.
- **Standard-IA:** Infrequent access, lower cost.
- **One-Zone-IA:** Single AZ storage, lower cost, reduced availability.
- **Glacier:** Long-term archival with variable retrieval times.
- **Intelligent-Tiering:** Automatically moves objects between two access tiers.

S3 Security

- **Bucket Policies:** Define permissions using JSON.
- **ACLs:** Legacy method, basic read/write permissions.
- **MFA Delete:** Requires multi-factor authentication to delete objects.
- **Server-Side Encryption:** Automatic encryption of data.
- **Client-Side Encryption:** Encrypts data before uploading.



S3 CheatSheet

S3 Access and Management

- **Transfer Acceleration:** Fast data transfer using CloudFront edge locations.
- **Presigned URLs:** Temporary URLs for accessing objects.
- **Access Points:** Network endpoints with specific permissions and network controls.
- **Object Lambda:** Modify and process S3 data on the fly.

S3 Data Management

- **Lifecycle Policies:** Automate transitions between storage classes and deletions.
- **Replication:** Cross-region and same-region replication for data redundancy.
- **Batch Operations:** Perform large-scale batch operations on S3 objects.
- **Inventory:** Track and audit objects in an S3 bucket.

S3 Monitoring and Analytics

- **CloudWatch Metrics:** Monitor storage and request metrics.
- **S3 Storage Lens:** Analyze storage usage and find cost-optimization opportunities.
- **Event Notifications:** Integrate S3 events with other AWS services.

S3 for Specific Use Cases

- **Static Website Hosting:** Serve static content directly from S3 buckets.
- **Requesters Pay:** Charge requesters for accessing data.
- **Multi-Region Access Points:** Route requests to the closest bucket for low latency.

API CheatSheet

AWS API Overview

What is an API?

- Software allowing two applications/services to communicate.
- Most common type: HTTP/S requests.

AWS API Interaction:

- HTTPS requests with applications like Postman.
- Each AWS service has its own Service Endpoint.
- Authorization via signed requests and tokens.

AWS Developer Tools

- AWS Management Console: Web-based interface.
- AWS SDK: Interact using various programming languages.
- AWS CLI: Terminal/shell-based interaction. Requires Python for installation.

CLI CheatSheet

AWS CLI Essentials

What is a CLI? Processes commands in text form.

Important Components:

- Terminal: Text-only input/output environment.
- Shell: Program for inputting commands (e.g., Bash, Zsh, PowerShell).
- Console: Physical computer for input.

Access Keys:

- Required for programmatic AWS access.
- Never share or commit to a codebase.
- Store in `~/.aws/credentials`.

Interacting with AWS API

- Direct Interaction: HTTP requests.
- Common Tools: AWS Management Console, AWS SDK, AWS CLI.

AWS CLI Features

Command Structure:

- Commands map to service endpoints.
- Subcommands map to API actions.

CLI CheatSheet

Command Flags: Used for options and parameters. Flags are written after the subcommand.

Wait Commands: Wait for specific conditions (e.g., `instance-running`).

Aliases: Shorten commonly used commands. Configured in `~/.aws/cli/alias`.

Filtering and Querying: Filter data server-side or client-side. Use flags like `--filter`, `--query`.

AWS CLI Configuration

Configuration Files:

- `~/.aws/credentials`: Sensitive credentials.
- `~/.aws/config`: General configurations.

Named Profiles:

- Switch between different configurations.
- Use `--profile` flag.

CLI CheatSheet

Security and Access Management

Security Token Service (STS):

- Temporary, limited-privilege credentials.
- Common API actions: ``AssumeRole``, ``GetSessionToken``.

Signing AWS API Requests:

- AWS Signature Version 4 is the most common.

Endpoints and Networking

Service Endpoints:

- Global, regional, FIPS, dualstack endpoints.
- Troubleshooting tools: ``telnet``, ``nslookup``, ``ping``.

Data Formats and Tools

- JSON: Lightweight data-interchange format.
- YAML: Indent-based markup language.
- JQ Library: Parse and format JSON.
- YQ Library: Wrapper for JQ to handle YAML.

CLI CheatSheet

AWS CLI and Docker

AWS CLI Docker Image:

- Isolated container for AWS CLI commands.
- Challenges: Passing credentials, downloading files.

AWS CLI – Installing and Updating

Supported Systems: Windows, macOS, Linux. Prefer official AWS documentation for installation.

AWS CLI – MFA and Roles

- **MFA:** Protect credentials with multi-factor authentication. Requires temporary sessions for commands.
- **IAM Roles:** Use IAM roles for enhanced security and temporary credentials.

Environment Variables

- **Common Variables:** `AWS\_ACCESS\_KEY\_ID`, `AWS\_SECRET\_ACCESS\_KEY`, `AWS\_DEFAULT\_REGION`.
- Override configuration files and CLI parameters.

Autoprompt and Autocompletion

AWS CLI Autoprompt:

- Interactive shell for command completion.
- Modes: Full, Partial.
- Features: Fuzzy search, parameter completion, resource completion.

Amazon Virtual Private Cloud (VPC) CheatSheet

Introduction to VPC

VPC Definition: A logically isolated section of the AWS Cloud to launch AWS resources in a virtual network you define.

Components:

- **Virtual Machines:** EC2 instances.
- **Virtual Network Cards:** Elastic Network Interfaces (ENIs).
- **CLI Commands:** All VPC commands are under ``aws ec2``.

Core Components

- **Route Tables:** Routes traffic within a VPC
- **Virtual Private Gateway (VPN Gateway):** Connects VPC to a private external network
- **Internet Gateway (IGW):** Connects VPC to the internet
- **NAT Gateway:** Allows private instances to connect to services outside the VPC
- **Subnets:** Public (with public IPs) and Private (without public IPs)
- **Security Groups (SGs):** Stateful virtual firewalls at the instance level
- **Network Access Control Lists (NACLs):** Stateless virtual firewalls at the subnet level
- **VPC Endpoints:** Private connections to AWS support services
- **VPC Peering:** Connects VPCs to other VPCs

Amazon Virtual Private Cloud (VPC) CheatSheet

Key Features

Region Specific: VPCs do not span regions

Peering: Connect VPCs across regions

Limits:

- Up to 5 VPCs per region (adjustable)
- 200 subnets per VPC
- Up to 5 IPv4 and IPv6 CIDR blocks per VPC (adjustable to 50)

Cost: Most components are free, but some (e.g., NAT Gateway, VPC Endpoints) incur charges

Default VPC

Configuration:

- **IPv4 CIDR block:** 172.31.0.0/16
- **Subnet:** Size of /20 for each Availability Zone
- Internet Gateway, default Security Group, NACL, DHCP options set, Route Table with IGW route

Management: Default VPC can be deleted and recreated



NACLs and Security Groups CheatSheet

NACLs

Definition: Acts as a **stateless** firewall for subnets

Rules:

- Inbound and Outbound
- Allow and Deny
- Rule # determines order of evaluation

Use Cases: Blocking specific IP addresses, additional security measures

Security Groups

Definition: **Stateful** firewall at the instance level

Rules:

- Only Allow rules
- No Deny rules

Use Cases: Allow specific IP ranges, nested security groups

Limits:

- 10,000 Security Groups per region
- 60 inbound and outbound rules per SG
- 16 SGs per Elastic Network Interface (ENI)

Amazon Virtual Private Cloud (VPC) CheatSheet

Route Tables

Definition: Determines where network traffic is directed

Types:

- Main Route Table (default)
- Custom Route Table

Routes: Local, IGW, VPG, NAT Gateway, Instance, ENI, etc.

Gateways

Types:

- **Internet Gateway (IGW):** Public traffic
- **Egress-Only Internet Gateway (EO-IGW):** Outbound IPv6 traffic only
- **Virtual Private Gateway:** VPN connection endpoint

Features:

- Supports IPv6 and IPv4
- NAT for instances with public IPv4 addresses

VPC Peering

Definition: Connects VPCs over a direct network route

Features:

- IPv4 and IPv6 supported
- No single point of failure
- Data transfer charges apply for cross-AZ or cross-Region

Amazon Virtual Private Cloud (VPC) CheatSheet

Elastic IPs

Definition: Static IPv4 addresses

Use Cases:

- Persistent IP address for EC2 instances
- Fail-over instances

Charges: \$1 per allocated but unassociated IP

Network Address Usage (NAU)

Definition: Metric for planning and monitoring VPC size

Limits:

- VPC: 64,000 NAUs (default), 256,000 NAUs (quota increase)
- Peered VPCs: 128,000 NAUs (default), 512,000 NAUs (quota increase)

Shared VPCs

Definition: Allows sharing VPCs with other AWS accounts within the same organization

Benefits: Reduces number of VPCs, separate billing and access control

Amazon Virtual Private Cloud (VPC) CheatSheet

Direct Connect

Definition: Dedicated network connections from on-premises locations to AWS

Options:

- Lower Bandwidth: 50MBps-500MBps
- Higher Bandwidth: 1GBps, 10GBps, 100GBps

Requirements: Single-mode fiber, VLAN encapsulation, BGP support

VPNs

Types:

- **Site-to-Site VPN:** Connects VPC to on-premises network
- **Client VPN:** Securely connects users to AWS or on-premises networks

Components:

- VPN Connection, VPN Tunnel, Customer Gateway (CGW), Virtual Private Gateway (VGW), Transit Gateway

Jumpbox/Bastion

Definition: Security-hardened EC2 instances for accessing private subnet resources

Features: Provides secure access via SSH or RCP

VPC Lattice

Definition: Fully managed application networking service for connecting services within VPC

Components: Service Network, Listener, Target Group, Service Directory



VPC Endpoint and VPC Flow Logs CheatSheet

VPC Endpoints

Types:

- **Interface Endpoints:** Elastic Network Interfaces for service traffic
- **Gateway Endpoints:** Reliable connectivity to S3 and DynamoDB

Features: Private connections without IGW, NAT, VPN, or Direct Connect

VPC Flow Logs

Definition: Captures IP traffic information through VPC

Scoping: VPC, Subnets, ENIs, Transit Gateway

Delivery: Amazon S3, CloudWatch Logs, Kinesis Data Firehose

Amazon Virtual Private Cloud (VPC) CheatSheet

Transit Gateway

Definition: Network transit hub for interconnecting VPCs and on-premises networks

Features:

- Supports up to 5000 VPC attachments
- Handles up to 50 Gbps of traffic per attachment

Uses: VPN and Direct Connect connections

Traffic Mirroring

Definition: Sends a copy of network traffic from a source ENI to a target

Uses: Security monitoring, deep packet inspection

Route 53 Resolver DNS Firewall

Definition: Protects against DNS exfiltration

Features: Monitor and control domain queries with allow and deny rules

AWS Network Firewall

Definition: Stateful, managed network firewall and IDS/IPS for VPCs

Uses: Filtering traffic, deep packet inspection, protocol detection

AWS Verified Access

Definition: Secure access to AWS resources without a VPN using trusted connected providers



IAM CheatSheet

IAM Overview

- Create and manage AWS users and groups
- Use permissions to allow or deny access to AWS resources
- **IAM Users:** End users logging into the console or interacting with AWS resources
- **IAM Groups:** Group users to share permission levels (e.g., Administrators, Developers, Auditors)
- **IAM Roles:** Grant AWS resources permissions to specific AWS API actions
 - Associate policies with a role, then assign it to an AWS resource
- **IAM Policies:** JSON documents granting permissions to users, groups, or roles
 - Types: Managed Policies, Customer Managed Policies, Inline Policies

Policy Elements

- **Version:** Policy language version (latest is 2012-10-17)
- **Statement:** Container for policy elements
- **Effect:** Allow or Deny
- **Action:** List of actions the policy allows or denies
- **Principal:** Account, user, role, or federated user
- **Resource:** The resource the action applies to
- **Condition:** Circumstances under which the policy grants permission



IAM CheatSheet

Principle of Least Privilege (PoLP)

- Provide the least amount of permissions needed
- Concepts: Just-In-Time (JIT), Just-Enough-Access (JEA), Risk-based adaptive policies

AWS Account Root User

- Special user with full access; cannot be deleted
- Root user should not be used for daily tasks
- Strongly recommended to enable MFA for the root user

IAM Password Policy

- Set minimum password requirements and rotation policies

IAM Access Keys

- Allow programmatic interaction with AWS services
- Two access keys per user

IAM Multi-Factor Authentication (MFA)

- Additional layer of security
- Can be enforced by administrators



IAM CheatSheet

IAM Temporary Security Credentials

- Temporary credentials for roles and identity federation
- Last from minutes to hours
- Basis for roles and identity federation

IAM Identity Federation

- Types: Enterprise (SAML, Custom Federation broker), Web (Amazon, Facebook, Google, OIDC)
- Allows linking of electronic identities across multiple systems

AWS Security Token Service (STS)

- Request temporary, limited-privilege credentials
- Key API actions: AssumeRole, AssumeRoleWithSAML, AssumeRoleWithWebIdentity, etc.

IAM Cross-Account Roles

- Grant users from different AWS accounts access to your resources
- Role creation involves a policy granting sts:AssumeRole access

AWS Single Sign-On (SSO)

- Create or connect workforce identities in AWS and manage access centrally
- Identity sources: AWS SSO, Active Directory, SAML 2.0 IdP



EC2 CheatSheet

Introduction to EC2:

- Highly configurable virtual server
- Resizable compute capacity; launch instances in minutes
- Core component for AWS services

Instance Selection:

- Choose OS via Amazon Machine Image (AMI)
- Instance Types:
 - General Purpose: balance of compute, memory, and networking (e.g., T2, T3, M5)
 - Compute Optimized: high-performance processors (e.g., C5)
 - Memory Optimized: fast performance for large data sets (e.g., R5, X1)
 - Storage Optimized: high read/write access to large datasets (e.g., I3, D2)
 - Accelerated Computing: hardware accelerators (e.g., P3, G4)

Pricing and Configurations:

- Pricing varies by instance type and size (e.g., t2.nano \$0.0065/hour)
- Configure with storage options: EBS, EFS
- Add security: Security Groups, Key Pairs, IAM Roles



EC2 CheatSheet

Cloud-Init and User Data:

- Cloud-init for cross-platform cloud instance initialization
- User Data to run scripts at first boot (e.g., install web servers)
- Meta-data and User-data for instance configuration

Instance Metadata:

- Access metadata via special endpoint (IPv4: <http://169.254.169.254/latest/meta-data/>)
- Two versions: IMDSv1 (request/response), IMDSv2 (session-oriented)
- Categories include ami-id, block-device-mapping, elastic-gpus, etc.

Instance Lifecycle:

- States: pending, running, stopping, stopped, shutting-down, terminated
- Actions: launch, stop, start, terminate, reboot, retire, recover

Instance Types Naming Convention:

- Instance family, generation, processor family, additional capabilities, size (e.g., c7gn.xlarge)



EC2 CheatSheet

Burstable Instances:

- Handle bursts of higher CPU utilizations
- Modes: Standard (baseline with bursts), Unlimited (sustain high CPU)
- Types: T2, T3, T3a, T4g

Instance Profile:

- Reference to IAM role for instance access without AWS credentials
- Single IAM role per Instance Profile; change requires reboot

Placement Groups:

- Types: Partition, Cluster, Spread
- Optimize instance placement for communication, performance, durability

System Logs and Console:

- Logs delivered to CloudWatch or viewed in EC2 Management Console
- Instance Console Screenshot for troubleshooting boot issues

Connectivity Options:

- EC2 Instance Connect, Sessions Manager, SSH Client, Serial Console, Fleet Manager Remote Desktop



EC2 CheatSheet

Hostnames:

- Hostname types: IP Name (IPv4), Resource Name (IPv6)
- Format varies by region and instance type

Default User Names:

- Varies by AMI (e.g., ec2-user for Amazon Linux, ubuntu for Ubuntu)

Amazon Linux:

- AWS managed Linux distribution (based on CentOS and Fedora)
- Versions: AL1 (EOL Dec 2023), AL2, AL2023
- Package management: yum for AL2, dnf for AL2023

Amazon Linux Extras and EPEL:

- AL2: use Amazon Linux Extras for additional packages
- EPEL for broader range of packages

Differences between AL2 and AL2023:

- AL2: Python 2.7, OpenSSL 1.0.2, gp2 volumes, Cronie installed
- AL2023: Python 3, OpenSSL 3, gp3 volumes, SELinux enabled by default, Cronie not installed



AMI CheatSheet

AMI Information:

- Template for root volume (EBS Snapshot or Instance Store template)
- Launch permissions for AWS accounts
- Block device mapping for attached volumes

Use Cases:

- Incremental changes to OS, application code, system packages
- Routine patching with security updates
- Managed revisions with Launch Configurations or Templates

AWS Marketplace:

- Purchase subscriptions to vendor-maintained AMIs
- Popular security-hardened AMIs (e.g., Center of Internet Security (CIS))

Choosing an AMI:

- Consider region, OS, architecture (32/64-bit), launch permissions, root device volume (Instance Store or EBS)

Boot Modes:

- **Legacy BIOS:** Traditional, no secure boot, for legacy OS/software
- **UEFI:** Modern, supports secure boot, faster startups, larger drives, pre-boot environment with UI and network capabilities



AMI CheatSheet

Elastic Network Adapter (ENA):

- Supports up to 100 Gbps network speeds
- Used in Nitro system instances for enhanced networking

Root Device Type:

- EBS-backed: Storage independent, data persists through stops/terminations
- Instance store-backed: Native volumes, data lost on stop/termination

Creating and Copying AMIs:

- Create from running or stopped EC2 instances
- Copy AMIs, including across regions, with optional encryption

Store and Restore AMIs:

- Store in S3 for long-term storage, lifecycle management, disaster recovery, auditing/compliance
- Restore from S3 for cross-AWS partition copying



AMI CheatSheet

Deprecate, Disable, and Deregister:

- **Deregister:** Prevent new instances, does not delete snapshots
- **Deprecate:** Mark a date to stop usage
- **Disable:** Temporarily prevent usage, can be re-enabled

Sharing AMIs:

- **Public:** Any AWS account can launch
- **Explicit:** Specific AWS accounts/organizations can launch
- **Implicit:** Only the owner can launch
- Can be sold in the AWS Marketplace

Virtualization Types:

- Hardware Virtual Machine (HVM): Hardware-assisted, supports unmodified OS, higher performance for direct hardware access
- Paravirtualization (PV): Software-assisted, requires OS modification, historically better performance but now less common

Auto Scaling Groups (ASG) CheatSheet

Introduction to ASG

- ASG contains EC2 instances managed as a group for automatic scaling.
- Automatic scaling methods: Capacity Settings, Manual Scaling, Health Check Replacements, Scaling Policies.
- ASGs work with ECS and EKS, but not Fargate.

ASG Use Case

1. Traffic hits the domain.
2. Route53 directs to load balancer.
3. Load balancer passes traffic to the target group.
4. Target group associated with ASG sends traffic to instances.
5. ASG Scaling Policy checks instance capacity.
6. Scaling Policy launches a new EC2 instance if needed.

Capacity Settings

- **Min Size:** Minimum running EC2 instances.
- **Max Size:** Maximum allowed EC2 instances.
- **Desired Capacity:** Ideal number of EC2 instances.
- Manual scaling involves changing these values.

Auto Scaling Groups (ASG) CheatSheet

Health Check Replacements

- **ELB Health Check:** ASG uses ELB health checks.
- **EC2 Health Check:** ASG replaces instances failing EC2 status checks.

ELB Integration

- ALB, NLB, GWLB are indirectly associated via Target Groups.
- Classic Load Balancers are directly associated with ASG.

Dynamic Scaling Policy

- **Adjustment Types:** ChangeInCapacity, ExactCapacity, PercentChangeInCapacity.
- **Policies:** Simple Scaling, Step Scaling, Target Tracking.

Simple Scaling Policy

- Adjust capacity when CloudWatch Alarm is triggered.
- Recommended to set a cooldown period.
- Prefer Step or Target Tracking policies.

Step Scaling Policy

- Adjust capacity at different thresholds when CloudWatch Alarm is repeatedly triggered.

Auto Scaling Groups (ASG) CheatSheet

Target Tracking Scaling Policy

- Scales capacity based on a target metric value (e.g., ASGAverageCPUUtilization).
- Automatically creates CloudWatch alarms.

Predictive Scaling Policy

- Analyzes historical load data to predict and scale based on traffic patterns.
- Uses last 14 days of data for hourly forecast.

Termination Policies

- **Predefined policies:** Default, AllocationStrategy, OldestLaunchTemplate, etc.
- Custom termination policies via Lambda Function.

Instance Refresh

- Replace instances to apply new configuration changes.
- **Features:** instance warmup, Min/Max healthy percentages, checkpoints, auto rollback.

Instance Scale-In Protection

- Prevents instances from being terminated.
- Can be enabled or disabled at creation or anytime.

Auto Scaling Groups (ASG) CheatSheet

Launch Template

- Includes parameters: AMI, Instance Type, Key Pair, Security Groups, etc.
- Required to make ASG work.
- Use SSM Parameter to avoid hardcoding ImageID.

Lifecycle Hooks

- Perform custom actions when an instance is pending or terminating.
- Notifies SQS or SNS, use `complete_lifecycle_action` to continue lifecycle.

Warm Pools

- Decrease latency for apps with long boot times.
- Instances in warm pool: Stopped, Running, or Hibernated.
- Use Lifecycle hooks for wait state.

Detach / Attach Instances

- Detach makes an instance independent.
- Attach criteria: running state, existing AMI, etc.
- Detach criteria: InService ASG state.

Auto Scaling Groups (ASG) CheatSheet

Suspend / Resume Processes

- Suspend processes for investigation or troubleshooting.
- Supports processes like Launch, Terminate, HealthCheck, etc.

Troubleshooting

- Failures: EC2 Instance Launch Failures, AMI Issues, Load Balancer Issues, Capacity Limits.
- Use AWS CLI to find exact error.



Elastic Load Balancing (ELB) CheatSheet

General Concepts:

- Load Balancer: Distributes incoming traffic to multiple targets (EC2, ECS, Fargate, EKS instances).
- Elastic Load Balancer (ELB): Suite of load balancers for traffic distribution.

Types of Load Balancers:

1. Classic Load Balancer (CLB):

- OSI Layer: 7 and 3/4
- Direct target attachment
- Legacy use
- Features: HTTP, HTTPS, TCP balancing, sticky sessions

2. Application Load Balancer (ALB):

- OSI Layer: 7 (HTTP/HTTPS)
- HTTP information routing, WAF support
- Use Cases: Microservices, e-commerce, SaaS applications
- Features: WebSockets, HTTP/2, authentication, WAF, ACM, Global Accelerator, CloudFront, Cognito



Elastic Load Balancing (ELB) CheatSheet

3. Network Load Balancer (NLB):

- OSI Layer: 3/4 (TCP/UDP)
- High throughput, low latency
- Use Cases: HPC, gaming, financial trading, IoT
- Features: Static IP, client IP preservation, Global Accelerator

4. Gateway Load Balancer (GWLB):

- Traffic routing to virtual appliances
- Security layer for traffic in transit

Traffic Management:

- Listeners: Evaluate incoming traffic based on port. Common ports: 443 (HTTPS), 80 (HTTP).
- Rules (ALB only): Invoked by listeners. Actions: forward to target groups. Conditions: HTTP request attributes (path, query, headers, etc.).
- Target Groups: Group of targets (EC2 instances, IP addresses). Not used in CLB.

Sticky Sessions (Session Affinity):

- Binds user session to specific compute target. Ensures all requests in a session go to the same instance. Typically used with CLB, can be enabled for ALB. Uses cookies for session persistence.



Elastic Load Balancing (ELB) CheatSheet

Cross-Zone Load Balancing:

- Distributes traffic evenly across instances in all Availability Zones. Default: Enabled for ALB and NLB, Disabled for CLB.

X-Forwarded-For (XFF) Header:

- Captures the original client IP address in HTTP proxy/load balancer scenarios.

Health Checks:

- Monitors instance status as InService or OutOfService. ALB and NLB use health checks on target groups.

Target Registration and Deregistration:

- Registration: Specific targets (EC2, IPs, Lambda functions).
- Deregistration: Connection draining to complete in-flight requests before removing target.

Mutual Authentication (mTLS):

- Supported via ELB Trust Stores using X509 Certificates.



Elastic Load Balancing (ELB) CheatSheet

Use Cases for Each Load Balancer:

1. Application Load Balancer (ALB):
 - Microservices: Efficiently route and manage API calls.
 - E-commerce: Handle HTTP/HTTPS traffic with security features.
 - Corporate Websites: Manage user authentication and web requests.
 - SaaS Applications: Ensure reliable user sessions and routing.
2. Network Load Balancer (NLB):
 - High-Performance Computing: Manage large-scale, low-latency TCP/UDP traffic.
 - Gaming Platforms: Support real-time multiplayer interactions.
 - Financial Trading: Ensure rapid, reliable transaction processing.
 - IoT Ecosystems: Connect numerous smart devices efficiently.

Advanced Features:

- Web Application Firewall (WAF): Protect ALB from common web exploits and vulnerabilities.
- AWS Certificate Manager (ACM): Manage SSL/TLS certificates for secure HTTP/HTTPS traffic.
- Global Accelerator and CloudFront: Enhance global availability and performance through optimized routing and caching.
- AWS Cognito: Authenticate and manage user identities securely.
- Mutual Authentication (mTLS): Implement mTLS for secure, authenticated connections between clients and servers.³⁶



Elastic Load Balancing (ELB) CheatSheet

Configuration and Best Practices:

- Setting Up Listeners and Rules: Listeners define ports for traffic evaluation. Rules specify conditions and actions for request handling (only ALB).
- Enabling Sticky Sessions: Use AWS CLI to enable application-based or duration-based stickiness. Ensure client supports cookies for session persistence.
- Cross-Zone Load Balancing Configuration: Enable or disable cross-zone balancing as needed (default varies by load balancer type).
- Health Checks: Regularly monitor instance health and adjust settings to maintain optimal traffic flow.
- X-Forwarded-For Header: Implement XFF header checking to trace client IP addresses in load-balanced environments.
- Target Management: Ensure proper registration and deregistration of targets to maintain service continuity and efficiency.



Route 53 CheatSheet

- **Route 53:** A Domain Name Service (DNS) integrated with AWS. Comparable to GoDaddy or NameCheap.
- **Register and manage domains:** Allows you to register new domains directly within AWS. Manage existing domains and renew them.
- **Create various record sets on a domain:** A record sets, CNAMEs, MX records, etc., to manage traffic routing.
- **Implement complex traffic flows:** Supports advanced routing configurations like blue/green deployments and failovers.
- **Monitor records via health checks:** Continuously checks the health of endpoints to ensure traffic is routed to healthy instances.
- **Resolve VPC's outside of AWS:** Route 53 can manage DNS queries for resources inside and outside of AWS.

Use Case

- Use Route 53 to point custom domains to AWS resources:
 1. **Incoming Internet Traffic:** Route 53 receives incoming requests and directs them to the appropriate AWS service.
 2. **Elastic Load Balancer (ALB):** Route traffic to your web applications behind an ALB for load balancing.
 3. **EC2 Instance:** Direct traffic to specific instances, useful for tasks like tweaking AMIs.
 4. **API Gateway:** Route traffic to API Gateway which handles API requests.
 5. **CloudFront:** Direct traffic to CloudFront for content delivery, often from S3 static-hosted websites.
 6. **Elastic IP (EIP):** Route traffic to static IPs, such as those hosting specific applications like a Minecraft server.



Route 53 CheatSheet

Hosted Zone: Container for record sets scoped to route traffic for a specific domain or subdomains.

- **Public Hosted Zones:** Route traffic inbound from the Internet.
- **Private Hosted Zones:** Route traffic within an Amazon VPC.

Building SaaS with Subdomains:

- Create a hosted zone domain.
- Create a record set pointing to the subdomain nameservers.
- Create a hosted zone for the subdomain (e.g., app).
- Create a wildcard record set to route traffic back to the root of the subdomain.

Record Sets

- **Record Sets:** Collection of records determining where to send traffic.
- **Batch Changes:** Always changed in batch via API.
- **Record Types Supported:**
 - A, AAAA, CAA, CNAME, DS, MX, NAPTR, NS, PTR, SOA, SPF, SRV, TXT
- **Action Types:**
 - CREATE: Create a new record set.
 - DELETE: Delete an existing record set.
 - UPSERT: Create or update a record set.



Route 53 CheatSheet

Traffic Flow

- **Visual Editor:** Create sophisticated routing configurations using various routing types.
- Supports versioning for updates roll-out or roll-back.
- Various routing types: Weighted, Failover, Geolocation, Latency, Multivalue Answer, Geoproximity.

Routing Policies

- **Simple Routing Policies:** Basic routing policy with one record and multiple IP addresses. Returns all values in random order.
- **Weighted Routing Policies:** Split traffic based on assigned weights. Direct certain percentage to one server, others to different servers.
- **Latency Based Routing:** Direct traffic based on lowest network latency. Requires latency resource record for each region.
- **Failover Routing Policies:** Active/passive setup with primary and secondary sites. Monitors health checks, fails over to secondary if primary is unhealthy.
- **Geolocation Routing Policies:** Direct traffic based on the geographic location of the request.
- **Geoproximity Routing Policies:** Direct traffic based on geographic location with bias value to expand/shrink traffic regions.
- **Multi-Value Answer Policies:** Return multiple values (e.g., IP addresses) for DNS queries. Similar to Simple Routing but includes health checks.



Route 53 CheatSheet

Health Checks:

- Check endpoint health every 30s (can reduce to 10s).
- Can initiate failovers if endpoint is unhealthy.
- Integrate with CloudWatch for alerts.
- Monitor other health checks to create chains of reactions.
- **Capacity:** Up to 50 health checks per AWS account

Route 53 Resolver

- **Amazon Route 53 Resolver:** DNS server for resolving queries between on-premise networks and VPCs.
- **Inbound Endpoints:** Resolve DNS queries to your VPC from on-premises or other VPCs.
- **Outbound Endpoints:** Resolve DNS queries from your VPC to on-premises networks or other VPCs.

Route 53 DNSSEC (Domain Name System Security Extensions)

- Suite of extensions for securing DNS data exchange.
- **Capabilities:** DNSSEC signing validates DNS responses and prevents tampering.
- **Setup:** Create a KSK signing key. Enable DNSSEC for the domain.



Route 53 CheatSheet

Route 53 Zonal Shift

- Capability in Route 53 Application Recovery Controller (ARC).
- Shifts load balancer resources from impaired to healthy Availability Zones.
- **Supported:** Application Load Balancers (ALB) and Network Load Balancers (NLB) without cross-zone load balancing.
- **Not Supported:** ALB as accelerator endpoint in AWS Global Accelerator.
- **Restrictions:** Only for specific load balancers in a single Availability Zone, not multiple zones.

AWS Global Accelerator CheatSheet

Overview

- AWS Global Accelerator: Optimizes path from end user to web servers.
- Edge Locations: Deploys Global Accelerator within Edge Locations.

Accelerator Types

- **Standard:** Automatically routes to the nearest healthy endpoint.
- **Custom Routing:** Routes to specific EC2 instances.

Components

- **Listeners (TCP 3000):** Listens for traffic on a specific port. Sends traffic to an endpoint group.
- **Endpoint Groups:** Collection of endpoints within a specific AWS Region.
 - **Traffic dial:** Adjusts the percentage of traffic.

Endpoint Types

- Network Load Balancer
- Application Load Balancer
- EC2 Instances
- Elastic IP Addresses



CloudFront CheatSheet

- **Content Delivery Network (CDN):** A distributed network of servers that delivers web pages and content to users based on their geographical location, the origin of the webpage, and a content delivery server.
- **CloudFront:** A CDN used to deliver:
 - Static content
 - Dynamic content
 - Streaming videos
 - Web sockets
- **Additional Features:**
 - Fronted with AWS WAF for OWASP TOP 10 protection.
 - Streams Videos On Demand using ISS Microsoft Smooth Streaming.

CloudFront - Core Components

- **Origin:** Location where all original files are located (e.g., S3 Bucket, EC2 Instance, ELB, Route53).
- **Edge Location:** Compute located strategically close to end-users.
- **Regional Caches:** Compute located in broad geographic locations to speed up requests for edge locations.
- **Distribution:** Collection of Edge locations and Regional Cache that defines how cached content should behave.



CloudFront CheatSheet

CloudFront - Lambda@Edge

- **Lambda@Edge:** Lambda functions to override the behavior of requests and responses.
- **Functions Deployed at:** Regional Edge Caches.
- **4 Functions for Lambda@Edge:**
 1. **Viewer request:** When CloudFront receives a request from a viewer.
 2. **Origin request:** Before CloudFront forwards a request to the origin.
 3. **Origin response:** When CloudFront receives a response from the origin.
 4. **Viewer response:** Before CloudFront returns the response to the viewer.
- Supported Languages: Python, Node.js.
- **Use Cases:**
 - **Viewer Request:** Redirect HTTP to HTTPS, inspect cookies for user authentication, modify headers for A/B testing. **Eg:** Redirect HTTP to HTTPS.
 - **Viewer Response:** Add security headers, set cookies for client-side tracking, customize error messages. **Eg:** Add security headers (e.g., CSP, HSTS).
 - **Origin Request:** Rewrite URLs for SEO or routing, inject headers for origin authentication, selective content serving based on user-agent. **Eg:** Serve different versions based on user-agent.
 - **Origin Response:** Modify headers to control caching, update URLs in HTML for versioning, customize error responses from the origin. **Eg:** Change 404 to 200 and provide custom response body.



CloudFront CheatSheet

CloudFront Functions

- Lightweight edge functions for high-scale, latency-sensitive CDN customizations.
- **Advantages:** Cheaper, faster, but more limited than Lambda@Edge functions.
- **Functions Deployed at:** Edge Locations.
- **2 Functions for CloudFront Functions:**
 1. **Viewer request:** When CloudFront receives a request from a viewer. Eg: Redirect HTTP to HTTPS.
 2. **Viewer response:** Before CloudFront returns the response to the viewer. Eg: Add security headers
- **Supported Language:** JavaScript (ECMAScript 5.1 compliant).

Use Cases

- Cache key normalization
- Header manipulation
- Status code modification and body generation
- URL redirects or rewrites
- Request authorization



EBS CheatSheet

Introduction to EBS

- **IOPS (Input/Output Per Second)**

- Speed of non-contiguous reads and writes on a storage medium.
- High I/O = lots of small, fast reads and writes.

- **Throughput**

- Data transfer rate to and from the storage medium in megabytes per second.

- **Bandwidth**

- Measurement of the total possible speed of data movement along the network.
- Think of Bandwidth as the Pipe and Throughput as the Water.

EBS Volumes

- **General Purpose SSD (gp2)**

- **Use:** General workloads, transactional workloads, virtual desktops, medium-sized databases, low-latency applications, boot volumes, development, and test environments.
- **Durability:** 99.8% - 99.9%.
- **Volume Size:** 1 GiB – 16 TiB.
- **Max IOPS:** 16,000.
- **Max Throughput:** 250 MiB/s.



EBS CheatSheet

- **General Purpose SSD (gp3)**
 - **Use:** Similar to gp2 but with up to 20% lower cost per GB.
 - **Durability:** 99.8% - 99.9%.
 - **Volume Size:** 1 GiB – 16 TiB.
 - **Max IOPS:** 16,000.
 - **Max Throughput:** 1,000 MiB/s.
- **Provisioned IOPS SSD (io1)**
 - **Use:** Sustained IOPS performance or more than 16,000 IOPS, I/O-intensive database workloads.
 - **Durability:** 99.8% - 99.9%.
 - **Volume Size:** 4 GiB – 16 TiB.
 - **Max IOPS:** 64,000.
 - **Max Throughput:** 1,000 MiB/s.
- **Provisioned IOPS SSD (io2)**
 - **Use:** More durable than io1, sustained IOPS performance.
 - **Durability:** 99.999%.
 - **Volume Size:** 4 GiB – 64 TiB.
 - **Max IOPS:** 256,000.
 - **Max Throughput:** 4,000 MiB/s.



EBS CheatSheet

- **io2 Block Express**

- **Use:** Higher throughput and IOPS, larger storage capacity.
- **Durability:** 99.999%.
- **Volume Size:** 4 GiB – 64 TiB.
- **Max IOPS:** 256,000.
- **Max Throughput:** 4,000 MiB/s.

- **Cold HDD (sc1)**

- **Use:** Throughput-oriented storage for infrequently accessed data.
- **Durability:** 99.8% - 99.9%.
- **Volume Size:** 125 GiB – 16 TiB.
- **Max IOPS:** 250.
- **Max Throughput:** 250 MiB/s.



EBS CheatSheet

- **Throughput Optimized HDD (st1)**
 - **Use:** Big data, data warehouses, log processing.
 - **Durability:** 99.8% - 99.9%.
 - **Volume Size:** 125 GiB – 16 TiB.
 - **Max IOPS:** 500.
 - **Max Throughput:** 500 MiB/s.
- **Magnetic (standard)**
 - **Use:** Workloads where data is infrequently accessed.
 - **Durability:** N/A.
 - **Volume Size:** 1 GiB – 1 TiB.
 - **Max IOPS:** 40-200.
 - **Max Throughput:** 40-90 MiB/s.



EBS CheatSheet

EBS - Moving Volumes

- **To move EBS volumes:**
 - **From one Availability Zone (AZ) to another:**
 1. Take a Snapshot of the volume.
 2. Create an AMI from the Snapshot.
 3. Launch a new EC2 instance in the desired AZ.
 - **From one Region to another:**
 1. Take a Snapshot of the volume.
 2. Create an AMI from the Snapshot.
 3. Copy the AMI to another region.
 4. Launch a new EC2 instance from the copied AMI.

EBS - Encrypted Root Volume

- **Encryption:**
 - EBS Root Volumes are not encrypted by default.
 - Encryption must be turned on at the time of creating an EBS volume or an EC2 instance



EBS CheatSheet

Steps to encrypt an existing volume:

1. Take a Snapshot of the unencrypted volume.
2. Create a copy of the Snapshot and select the Encryption option.
3. Create a new AMI from the encrypted Snapshot.
4. Launch a new EC2 instance from the created AMI.



EBS CheatSheet

EBS Volumes vs Instance Store Volumes

- **EBS Volumes:**

- Durable, block-level storage device that can attach to a single EC2 instance.
- Created from an EBS Snapshot.
- Can start and stop instances, with data persisting if you reboot your system.
- Ideal for when you want data to persist (most use cases).

- **Instance Store Volumes (Ephemeral):**

- Temporary storage type located on disks physically attached to a host machine.
- Created from a template stored on the host machine.
- Cannot stop instances using Instance Store Volumes; only terminate.
- Data is lost if the host fails or the instance is terminated.
- Ideal for temporary backup and storing application cache, logs, or other random data.



EBS CheatSheet

Storage Volumes – HDD

- **Hard Disk Drive (HDD)**

- Uses rotating platters, an actuator arm, and a magnetic head.
- Good for continuous data writing but not for small reads/writes.
- RPMs (Revolutions Per Minute):
 - 7200 RPM: Standard for desktops, good balance of performance and cost.
 - 5400 RPM: Used in laptops and applications with low power consumption.
 - 10000 RPM: High-performance, less common due to SSD rise.

- **HDD RAID Configurations**

- RAID 0 (Striping): No redundancy, data split across disks, minimum 2 disks.
- RAID 1 (Mirroring): Data duplicated, high redundancy, minimum 2 disks.
- RAID 5 (Striping with Parity): Combines striping and parity, minimum 3 disks.
- RAID 6 (Striping with Double Parity): Similar to RAID 5 with double parity, minimum 4 disks.
- RAID 10 (1+0): Combination of RAID 1 and RAID 0, minimum 4 disks.



EBS CheatSheet

Storage Volumes – SSD

- **Solid State Drive (SSD)**

- Uses integrated circuit assemblies, typically using flash memory.
- Resistant to physical shock, quick access time, no moving parts.

- **Types of SSDs**

- SATA SSDs: Most common, good performance, limited by SATA interface.
- NVMe SSDs: Higher performance, uses PCIe interface, available in M.2 and PCIe card forms.
- M.2 SSDs: Compact, suitable for laptops, can use SATA or NVMe interfaces.
- U.2 SSDs: Similar to M.2 but designed for 2.5-inch drive bays, used in enterprise environments.
- Portable SSDs: External drives for easy portability, connected via USB or Thunderbolt.
- PCIe SSDs: Add-on cards for high performance, fits into motherboard PCIe slots.

- **PCIe Slot Generations and Bandwidth**

- PCIe 2.0: 2007, 500 MB/s per lane, 8 GB/s x16 slot.
- PCIe 3.0: 2010, 1 GB/s per lane, 16 GB/s x16 slot.
- PCIe 4.0: 2017, 2 GB/s per lane, 32 GB/s x16 slot.
- PCIe 5.0: 2019, 4 GB/s per lane, 64 GB/s x16 slot.
- PCIe 6.0: 2022, 8 GB/s per lane, 128 GB/s x16 slot.



EBS CheatSheet

Storage Volumes - Magnetic Tape

- A large reel of magnetic tape used for data storage.
- Medium and large-sized data centers use both tape and disk formats.
- Typically comes in the form of a cassette.
- **Advantages:** Durable for decades (up to 30 years). Cheap to produce.



EFS CheatSheet

Elastic File System (EFS): File storage service for EC2 instances.

Features:

- Storage capacity grows (up to petabytes) and shrinks automatically based on data stored.
- Multiple EC2 instances in the same VPC can mount a single EFS Volume.
- EC2 instances install the NFSv4.1 client to mount the EFS volume.
- EFS uses the Network File System version 4 (NFSv4) protocol.
- EFS creates multiple mount targets in all your VPC subnets.

Amazon EFS Client

amazon-efs-utils: Open-source tools for EFS, includes mount helper.

Features:

- Enables Amazon CloudWatch monitoring.
- Mounting on supported EC2 instances (Linux/Mac) with IAM authorization and access points.
- Auto-mounting on EC2 reboot and instance creation.
- On-premise Linux client support.

Limitations: Does not support EC2 Windows instances; previously used standard Linux NFS client.



EFS CheatSheet

Mounting EFS

- **Mount helper** defines a new network file system type, **efs**, compatible with the standard mount command in Linux.
- **Automating mount:** Use `/etc/fstab` configuration with `netdev` option.
- Mount using:
 - File system DNS name.
 - File system ID.
 - Mount target IP address.

EFS Mount Options

- `nfsvers=4.1`: EC2 Linux instances.
- `nfsvers=4.0`: EC2 Mac (macOS Big Sur, Monterey, Ventura).
- `rsiz=1048576`, `wsiz=1048576`: Max bytes for READ/WRITE to avoid performance issues.
- `hard`: Indefinite retries for data integrity.
- `timeo=600`: 60-second NFS request timeout.
- `retrans=2`: Number of retries before recovery.
- `noresvport`: Uses non-privileged TCP source port after reconnection.
- `mountport=2049`: EC2 Mac (macOS Big Sur, Monterey, Ventura).

Amazon FSx CheatSheet

Amazon FSx

- High-performance file systems in the cloud supporting various protocols:
 - **NetApp ONTAP**: Enterprise storage for petabytes of data
 - **OpenZFS**: Open-source storage, developed by Sun Microsystems
 - **Windows File Server (WFS)**: Native Windows file storage with SSD/HDD support, integrates with Active Directory
 - **Lustre**: Open-source, optimized for parallel computing

Amazon FSx for Windows File Server (WFS)

- **Features**: Windows compatibility, sub-ms latencies, enterprise performance
- **Use Cases**: Business apps, home directories, web/content management, data analytics, software builds, media processing
- **Requirements**: EC2/Workspace Instance, AppStream 2.0, VMWare Cloud on AWS

Amazon File Cache

- High-speed cache for datasets from on-premises/AWS/S3, sub-ms latencies, high throughput
- **Accessible**: EC2, ECS, EKS
- **Compatible AMIs**: Amazon Linux, RHEL, CentOS, Rocky Linux, Ubuntu
- **Integrations**: AWS Batch, Thinkbox Deadline for creative rendering workloads



Snow Family CheatSheet

Purpose: Physical data transfer to/from the cloud when internet/private connections are slow, difficult, or costly.

Snowcone

- **Storage:** 8 TB HDD / 14 TB SSD
- **Ports:** 2 RJ45 (1/10 Gb), USB-C, USB-C PD
- **Features:** E Ink shipping label; Size: 227 x 148.6 x 82.65mm; Weight: 4.5 lbs (2.1 kg); 256-bit encryption, Not HIPAA compliant

Snowball Edge

- **Types:**
 - **Compute Optimized:** 28 TB NVMe SSD
 - **Storage Optimized:** 80 TB HDD / 210 TB NVMe
- **Features:** LCD display, Cluster (3 to 16 nodes); Data Transfer: NFSv3, v4, v4.1, S3 over HTTP/HTTPS; Size: 548 x 320 x 501mm Weight: 49.7 lbs (22.3 kg); HIPAA compliant

Snowmobile

Storage: 100 PB

Form: 45-foot shipping container on a semi-trailer truck

Security: GPS, alarm, 24/7 video, optional escort vehicle

Process: AWS assists with network connection and data transfer



EC2 Auto Scaling Groups CheatSheet

- An ASG is a collection of EC2 instances grouped for scaling and management
- Scaling Out is when adding servers
- Scaling In is when you remove servers
- Scaling Up is when you increase the size of an instance (eg. updating Launch Templates with a larger size)
- Size of an ASG is based on a **Min**, **Max**, and **Desired Capacity**
- **Target Scaling policy** scales based on when a target value for a metric is breached (eg. Average CPU Utilization exceeds 75%)
- **Simple Scaling** policy triggers a scaling when an alarm is breached
- **Scaling Policy with Steps** is the new version of Simple Scaling policy and allows you to create steps based on escalation alarm values.
- Desired Capacity is how many EC2 instances you want to ideally run
- An ASG will always launch instances to meet minimum capacity
- Health checks determine the current state of an instance in the ASG
- Health checks can be run against either an ELB or the EC2 instances
- When an Auto Scaling launches a new instance, it uses a Launch Template which holds the configuration values for that new instance eg. AMI, InstanceType, Role
- Launch Template can be updated using the AWS Management Console, AWS CLI, or SDKs



SNS CheatSheet

Pub/Sub Overview

- **Publish-Subscribe Pattern:**
 - Publishers send messages to an event bus.
 - Subscribers receive messages from the event bus.
- **Key Points:**
 - Publishers don't know their subscribers.
 - Subscribers get messages automatically (pushed, not pulled).
 - Messages and events are interchangeable terms.

Introduction to AWS SNS (Simple Notification Service)

- **Purpose:** Decouples microservices, distributed systems, and serverless applications.
- **Components:**
 - **Publishers:** AWS SDK, AWS CLI, CloudWatch, AWS Services.
 - **SNS Topic:** Logical access point for communication, supports message filtering and fanout.
 - **Subscribers:** Lambda, SQS, Email, HTTP/S.

SNS Sources

- Various AWS services can publish to SNS topics (e.g., AWS Athena, Amazon RDS, AWS CodeBuild, AWS Lambda, Amazon S3).

SNS Destinations

- **Application-to-Application (A2A):** Data Firehose, Lambda Functions, SQS Queue, HTTP/S endpoint, AWS Event Fork Pipelines.
- **Application-to-Person (A2P):** Mobile applications, SMS, Email, AWS Chatbot, PagerDuty.



SNS CheatSheet

SNS Topics

Functionality

- **Group Subscriptions:** Deliver to multiple protocols (email, SMS, HTTP/S).
- **Automatic Formatting:** Format messages according to the subscriber's chosen protocol.
- **Encryption:** Supported via KMS.

Types of SNS Topics

- **Standard:**
 - Throughput: High
 - Delivery: At least once (possibility of duplicates)
 - Ordering: No guarantees
 - Use-Case: High volume alerts, notifications
- **FIFO (First-In-First-Out):**
 - Throughput: Lower compared to Standard
 - Delivery: Exactly once (no duplicates)
 - Ordering: Guaranteed within a message group
 - Use-Case: Banking transactions, ordered data processing
- **Message Attributes:** Provide structured metadata about the message.
- **Batch Publishing:** Can publish up to 10 messages at a time. Reduces SNS costs by a factor of 10.
- **Create Subscription:** To receive messages, create a subscription.
- **Filter Policy** allows filtering of messages based on message attributes or body.



SNS CheatSheet

- **Message Data Protection**
 - Safeguards data using protection policies to audit, mask, redact, or block sensitive information.
 - Scans for Personally identifiable information (PII) and Protected health information (PHI)
- **Raw Message Delivery:** Direct message delivery without JSON formatting
- **Delivery Policy:** Defines how SNS retries message delivery on errors.
- **Dead Letter Queue:** Send failed message attempts to an SQS queue.

Application as Subscriber

- Push Notifications: Sent to mobile apps as alerts, badge updates, or sound alerts.
- Supported Protocols
 - HTTP/HTTPS: Webhooks.
 - Email/Email-JSON: Notifications via email.
 - Amazon SQS: Messages to SQS queue.
 - AWS Lambda: Triggers Lambda functions.
 - SMS: Text messages.
 - Platform Application Endpoint: Mobile push notifications



Simple Queue Service (SQS) CheatSheet

- **Messaging System:** Asynchronous communication for decoupling processes; involves a sender (producer) and receiver (consumer).
- **Queueing System:** Deletes messages once consumed; not real-time, requires message pulling.
- **Simple Queueing Service (SQS):** Fully managed service for decoupling and scaling microservices, distributed systems, and serverless applications. Use case: Queueing transaction emails (e.g., signup, reset password).
- **Queue Types:**
 - **Standard:** Nearly unlimited transactions per second (tps); at least one delivery guarantee.
 - Consumers handle out-of-order and duplicate messages.
 - Best-effort ordering.
 - **FIFO:** Guarantees message order with exactly-once processing.
 - Requires unique Message Group ID.
 - Limited to 300 TPS; up to 10 messages read at a time.
 - **High Throughput:** Enabled for 3,000 messages per second with batching; 10x greater than standard FIFO
- **Message Size:** 1 byte to 256 KiB; larger messages require a library.
- **Message Retention:** Default 4 days; adjustable from 60 seconds to 14 days.
- **Encryption:** Using Amazon managed Server-Side Encryption (SSE) or KMS.
- **Extended Client Libraries:** For Java/Python, supporting messages up to **2 GB**. Saves payload to S3 bucket; stores reference in SQS.



Simple Queue Service (SQS) CheatSheet

- **ABAC (Attribute-Based Access Control):** Control access based on tags and aliases.
- **Access Policy:** Common actions are Send, Receive, and Delete messages
- **Visibility Timeout:** Prevents re-processing, 0 to 12 hours (default 30s)
- **Delay Queues:** Postpone delivery, 0 to 15 minutes.
- **Message Timers:** Initial invisibility for individual messages. Not for FIFO queues.
- **Temporary Queues:** High-throughput, cost-effective, auto-deleted. For specific threads or processes.

Polling

- **Short Polling:** Immediate returns.
- **Long Polling:** Waits for message or timeout, reduces costs.



Service Catalog *CheatSheet*

- **AWS Service Catalog** enables organizations to **create and manage catalogs of products** that are **approved for use** on AWS to achieve consistent governance and meet compliance requirements.
- There are two types of Service Catalog users:
 1. **Catalog Administrator**: They managed the catalog
 2. **End Users**: They use the catalog
- A **product** is a **CloudFormation template** that defines the resources that will be launched
 - You can create or associate an **AWS Budget** with a product
 - Once a product is created, it cannot be edited, only **deleted**.
 - Also, a product must be removed from the portfolio and not provisioned by a user in order to delete.
 - In order for products to be made visible to users, it needs to be added to a portfolio.
- A **Portfolio** is a **collection of products** with the following options:
 - **Constraints** can restrict how products are used.
 - **Permissions** determine who can see and launch products by associating either **Groups, roles, or users**
 - You cannot restrict certain products to certain users in a portfolio, you have to just make additional portfolios
- When a Product is launched, it's called a **Provisioned Product**
- You can attach **Service Actions** (Which are just SSM Documents) to a product so end-users can perform governed actions on the product.



CloudWatch *CheatSheet*

- The Pillars of Observability are **Metrics**, **Logs**, and **Traces**. You need to use all three together to obtain **Observability**
- AWS CloudWatch is a collection of monitoring services:
 - **CloudWatch Logs** — centralization log management service. Logs from AWS Services, Application Logs running on your EC2s
 - **CloudWatch Log Insights** — Robust way to filter your logs. It auto-detects log patterns, creates dynamic fields, and composes queries.
 - **CloudWatch Metrics** — Aggregates data points from AWS Service to produce a metric. (eg AvgCPUUtilization)
 - **CloudWatch Alarms** — Define a threshold up on a metric and reacts by triggering action such as an EC2, Auto Scaling, or SNS action
 - **CloudWatch Events (EventBridge)** — A serverless account-wide event bus. Create event-driven loosely coupled apps with AWS Services
 - **CloudWatch Dashboards** — Create dashboards filled with graphs powered by CloudWatch metrics
 - **CloudWatch Container Insights** — collect, aggregate, and summarize metrics and logs from your containerized apps and microservices
 - **CloudWatch Contributor Insights** — view the top contributors impacting the performance of your systems and applications in real-time
 - **CloudWatch Synthetics** — Test your web apps to see if they're broken. Uses Puppeteer underneath.
 - **CloudWatch Service Lens** — "Observability" for distributed or serverless apps by pulling X-Ray Traces, CloudWatch Logs, and Metrics



CloudWatch Logs *CheatSheet*

CloudWatch Logs is used to **monitor**, **store**, and **access** your log files

- Export logs to S3 to analyze with Athena
- Stream logs to ElasticSearch Service to run a full-text search
- Stream CloudTrail Events to CloudWatch Logs to allow you to react to CloudTrail Events
- Encrypted by default, You can apply your own KMS Key
- Retains logs indefinitely by default. You can choose a custom retention period between 1 Day and 10 years
- Support a simple Filtering Syntax
- **Log Groups** — A container for the collection of log streams. Uses the forward slash naming convention (eg. /my-app/prod/us-east/)
- **Log Streams** — Represent a log, usually created by AWS Services, which can be manually created
- **Log Events** — Represents a single event (a single line from the log file)
- In order to send custom application logs running on your EC2 instance, you need to install the **SSM CloudWatch Agent**
- When the agent is installed, you update the **/etc/awslogs/awslogs.conf** on your EC2 instance to include the logs you want to send to CloudWatch



CloudWatch Logs Insights *CheatSheet*

CloudWatch Logs Insights enables you to **interactively search and analyze your CloudWatch log data**

- CloudWatch Logs Insights supports all kinds of logs
- CloudWatch Logs Insights is generally used from the AWS Console to create ad-hoc queries
- CloudWatch Logs Insights has its own language called CloudWatch Logs Insights **Query Syntax**
- A single request can query up to **50 log groups**.
- Queries **time out after 15 minutes**, if they have not been completed.
- Query results are **available for 7 days**.
- CloudWatch Logs Insights comes with a bunch of sample queries
- CloudWatch Logs Insights analyzes the log events in imported logs to generate **Dynamic Fields**
- CloudWatch Logs Insights always generates out 5 of the following fields:
 - **@message** — the raw unparsed log event.
 - **@timestamp** — the event timestamp contained in the log event's timestamp field.
 - **@ingestionTime** — the time when the log event was received by CloudWatch Logs.
 - **@logStream** — the name of the log stream that the log event was added to.
 - **@log** — is a log group identifier in the form of account-id:log-group-name.
- CloudWatch auto-discovery fields for the following AWS Services
 - Amazon VPC Flow Logs
 - Amazon Route53
 - AWS Lambda
 - AWS CloudTrail
- JSON logs keys will all become discovered fields
- For other logs that are not automatically discovered, you can use the parse command to create ephemeral fields



CloudWatch Metrics *CheatSheet*

CloudWatch Metrics represents a **time-ordered set of data points**. It's a **variable** that is **monitored over time**.

- Metrics are data about the performance of your systems.
- CloudWatch comes with many **predefined** metrics that are generally name-spaced by AWS Service.
- Very common predefined metrics at the EC2 Per-Instance Metrics: CPUUtilization, DiskReadOps, DiskWriteOps, DiskReadBytes, DiskWriteBytes, NetworkIn, NetworkOut, NetworkPacketsIn, NetworkPacketsOut
- Metric data is kept for 15 months, enabling you to view both up-to-the-minute data and historical data.
- **Metric math** enables you to **query multiple CloudWatch metrics** and **use math expressions** to create new time series
- You can publish your own metrics using the AWS CLI or AWS SDK (eg. `aws cloudwatch put-metric-data`)
- You can define custom metrics in two different resolutions:
 - standard resolution (1 minute)
 - high resolution (> 1 minute to 1 second)
- With High-Resolution, you can track in intervals of: 1 second, 5 seconds, 10 seconds, 30 seconds, multiple of 60 seconds.
- Data availability (when you can see the data) varies based on the service
 - **EC2 Basic Monitoring (5 minutes)**
 - EC2 Detailed Monitoring (1 minute)
 - Other Services generally 1 minute but could also be 3 or 5 minutes
- You can turn on EC2 Detailed Monitoring for a price
- Not all EC2 metrics are tracked by default. You have to install the CloudWatch Agent to collect
 - **Memory utilization** (Am I running out of memory?)
 - Disk Swap utilization
 - **Disk Space utilization** (Am I running out of disk space?)
 - Page file utilization



CloudWatch Alarms *CheatSheet*

A **CloudWatch Alarm** monitors a **CloudWatch Metric** based on **a defined threshold**.
When an alarm **breaches** (goes outside the defined threshold), then it changes **state**.

Metric Alarm States

- **OK** The metric or expression is **within** the defined threshold
- **ALARM** The metric or expression is **outside** of the defined threshold
- **INSUFFICIENT_DATA**
 - The alarm has **just started**
 - the metric is **not available**
 - **Not enough data** is available

When it changes state, we can define what **action it should trigger**:

- Notification
- Auto Scaling Group
- EC2 Action
- You can define Conditions of either a **Static** or **Anomaly Detection**
 - **Static** set a static value as the threshold (eg. 100 USD)
 - **Anomaly Detection** sets a band around the data points, helps prevent false positives, and is more flexible if you have seasonal data
- **Composite Alarms** — Allows you to watch multiple alarms and require both to trigger before resulting in an alarm action
 - The alarms being watched must have no actions sets
 - You can only trigger an SNS as the action (so no EC2 or ASG actions)



Lambda CheatSheet

- **Lambdas** are serverless **functions**. You upload your code, and it runs without you managing or provisioning any servers.
- Lambda is **serverless**. You don't need to worry about the underlying architecture
- **Lambda is ideal for short-running tasks** where you don't need to customize the OS environment. For long-running tasks (> 15 minutes) and a custom OS environment, consider using **Fargate**.
- There are **7 runtime language environments** officially supported by Lambda: **Ruby, Python, Java, NodeJs, C#, Powershell, and Go**
- You pay **per invocation** (based on duration and memory used) rounded up to the nearest 100 milliseconds. The first 1M requests per month are free.
- You can adjust the **duration timeout** for up to **15 mins** and memory up to **10,240 MB**
- You can trigger Lambdas from the SDK or multiple AWS services (eg. S3, API Gateway, DynamoDB)
- By default, Lambdas run in **No VPC**. To interact with some services, you need to have your Lambda in the same VPC (eg. RDS)
- Lambda can scale to **1000 concurrent functions** in seconds. (1000 is the default, you can increase it with AWS Service Limit Increase)



Lambda CheatSheet

- **Lambdas have Cold Starts.** If a function has not been recently executed, there will be a delay. Recent improvements have focused on reducing cold start durations, especially for Provisioned Concurrency and API Gateway triggers
- **Advanced Logging Controls:** Capture logs in JSON format, adjust log levels, and select specific CloudWatch log groups for your Lambda functions.
- **Deprecation Notice:** Support for **Python 3.8** will end on October 14, 2024. Update your functions to Python 3.9 or later.
- **Runtime Management:** New controls make it easier to update runtime versions via the AWS Management Console or AWS CLI.
- **Container Images:** Deploy functions as container images for more flexibility in runtime environment and dependencies.



Kinesis CheatSheet

- **Amazon Kinesis** is the AWS solution for **collecting, processing, and analyzing streaming data** in the cloud. When you need **"real-time"**, *think* Kinesis.
- **Kinesis Data Streams**
 - Per running shard, data can persist within the stream. Data is in order, and every consumer keeps its position. Consumers have to be manually added (coded).
 - Data persists for **24 hours (default)**, up to **8760 hours (365 days)**
 - Inspect data records directly from the AWS Management Console without a dedicated consumer app.
- **Kinesis Firehose**
 - Pay for only the data ingested. Data **immediately disappear** once processed.
 - Consumer of choice is from a predefined set of services: S3, Redshift, OpenSearch Service, Splunk, Datadog, MongoDB, New Relic
 - Supports data transformation before delivering to destinations.
- **Kinesis Data Analytics** (Managed Service for Apache Flink):
 - Perform queries in real-time. Needs Kinesis Data Streams/Firehose as input and output.
 - Supports exactly-once processing semantics and improved connectors.
 - Includes support for Apache Flink version 1.15.



Kinesis CheatSheet

- **Kinesis Video Streams:**
 - Securely ingests and stores video and audio encoded data to consumers such as SageMaker, Rekognition, or other services to apply Machine Learning and video processing.
 - Edge recording and scheduled cloud streaming capabilities (preview).
- **KPL (Kinesis Producer Library):**
 - A Java library to write data to a stream.
 - AWS SDK can be used to write data to streams, but KPL is more efficient.



ElastiCache CheatSheet

- **ElastiCache** is a managed **in-memory** caching service
- ElastiCache can launch either **Memcached** or **Redis**
- **Memcached** is a simple key / value store preferred for caching HTML fragments and is arguably faster than Redis
- **Redis** has richer data types and operations. Great for leaderboard, geospatial data, or keeping track of unread notifications.
- Redis supports replication and clustering for higher availability and scalability.
- Redis offers data persistence options like RDB snapshots and AOF logs.
- A cache is a **temporary storage** area.
- Most frequent identical queries are stored in the cache
- Resources only **within the same VPC** may connect to ElastiCache to ensure low latencies.
- **ElastiCache supports encryption** in transit and at rest, and Redis Auth for securing clusters.
- **Redis supports Lua scripting** for complex operations and Redis Streams for handling message queues.



CloudTrail CheatSheet

- CloudTrail logs calls between AWS services
- **governance, compliance, operational auditing**, and **risk auditing** are keywords relating to CloudTrail
- When you need to know **who to blame**, *think* CloudTrail
- CloudTrail, by default, logs event data for the past 90s days via **Event History**
- To track beyond 90 days, you need to create a **Trail**
- To ensure logs have not been tampered with, you need to turn on **Log File Validation** option
- CloudTrail logs can be encrypted using **KMS (Key Management Service)**
- CloudTrail can be set to log across all AWS accounts in an Organization and all regions in an account.
- CloudTrail logs can be streamed to CloudWatch logs
- Trails are outputted to an S3 bucket that you specify
- CloudTrail logs three kinds of events: **Management events**, **Data events**, and **Insights events**
- **Management events** log management operations (eg. AttachRolePolicy)
- **Data Events** log data operations for resources (S3, Lambda) (eg. GetObject, DeleteObject, and PutObject)
- Data Events are **disabled** by default when creating a Trail.
- Trail which are logged within S3 can be analyzed using Athena



API Gateway CheatSheet

- API Gateway is a solution for creating secure APIs in your cloud environment at any scale.
- Create APIs that act as a front door for applications to access data, business logic, or functionality from back-end services.
- API Gateway throttles API endpoints at **10,000** requests per second (can be increased via service request through AWS support)
- **Stages** allow you to have multiple published versions of your API (eg. prod, staging, QA)
- Each Stage has an **Invoke URL**, which is the endpoint you use to interact with your API
- You can use a custom domain for your Invoke URL (eg. api.exampro.co)
- You need to publish your API via Deploy API. You choose which Stage you want to publish your API
- Resources are your URLs (eg. /projects)
- Resources can have child resources (eg. /projects/**-id-**/**edit**)
- You defined multiple Methods on your Resources (eg. GET, POST, DELETE)
- CORS issues are common with API Gateway, CORS can be enabled on all or individual endpoints
- Caching improves latency and reduces the number of calls made to your endpoint
- Same Origin Policies help to prevent XSS attacks
- Same Origin Policies ignore tools like Postman or curl
- CORS is always enforced by the client.
- You can require Authorization to your API via AWS Cognito or a custom Lambda.



RDS CheatSheet

- Relational Database Service (RDS) is the AWS Solution for relational databases.
- RDS instances are managed by AWS. You cannot SSH into the VM running the database.
- There are 6 relational database options currently available on AWS, Aurora, MySQL, MariaDB, Postgres, Oracle, Microsoft SQL Server
- Multi-AZ is an option you can turn on which makes an exact copy of your database in another AZ that is only standby
- For Multi-AZ, AWS automatically synchronizes changes in the database over to the standby copy
- Multi-AZ has Automatic Failover protection. If one AZ goes down, failover will occur, and the standby instance will be promoted to primary
- Read-Replicas allows you to run **multiple copies** of your database. These copies only allow **reads** (no writes) and are intended to alleviate the workload of your primary database to improve performance
- Read-Replicas use Asynchronous replication
- You must have automatic backups enabled to use Read Replicas



RDS CheatSheet

- You can have up to 5 read replicas
- You can combine Read Replicas with Multi-AZ
- You can have Read Replicas in another Region (Cross-Region Read Replicas)
- Replicas can be promoted to their own database, but this breaks replication
- You can have Replicas of Read Replicas
- RDS has 2 backup solutions: Automated Backups and Database Snapshots
- Automated Backups, you choose a retention period between 1 and 35 days. There is no additional cost for backup storage. You define your backup window
- Manual Snapshots, you manually create backups. If you delete your primary, the manual snapshots will still exist and can be restored
- When you restore an instance, it will create a new database. You just need to delete your old database and point traffic to the newly restored database
- You can turn on encryption at-rest for RDS via KMS



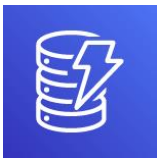
Aurora CheatSheet

- When you need a **fully-managed** Postgres or MySQL database that needs to scale, automatic backups, high availability, and fault tolerance, *think* Aurora
- Aurora can run MySQL or Postgres database engines
- Aurora MySQL is 5x faster than regular MySQL
- Aurora Postgres is 3x faster than regular Postgres
- Aurora is 1/10 the cost over its competitors with similar performance and availability options.
- Aurora replicates **6 copies** for your database across **3 availability zones**.
- Aurora is allowed up to **15 Aurora Replicas**
- An Aurora database can span multiple regions via **Aurora Global Database**
- **Aurora Serverless** allows you to stop and start Aurora and scale automatically while keeping costs low
- Aurora Serverless is ideal for new projects or projects with infrequent database usage



Redshift CheatSheet

- Data can be loaded from S3, EMR, DynamoDB, or multiple data sources on remote hosts.
- Redshift is a Columnar Store database that can SQL-like queries and is an OLAP.
- Redshift can handle petabytes worth of data. Redshift is for Data Warehousing
- Redshift's most common use case is Business Intelligence
- Redshift supports Multi-AZ deployments for higher availability
- Redshift can run via a single node or multi-node (clusters)
- Node sizes vary, with options available beyond 160 GB, tailored to different storage and performance needs
- A multi-node is comprised of a leader node and multiple compute nodes
- You are billed per hour for each node (excluding the leader node in multi-node)
- You are not billed for the leader node
- You can have up to 128 compute nodes
- Redshift has two kinds of Node Types: **Dense Compute** and **Dense Storage**
- Redshift ensures data durability by storing original data on compute nodes, automated backups on S3
- Similar data is stored on disk sequentially for faster reads
- Redshift database can be encrypted via KMS or CloudHSM
- Backup Retention is 1 day by default and can be increased to a maximum of 35 days
- Redshift can asynchronously back up your snapshot to Another Region delivered to S3
- Redshift uses Massively Parallel Processing (MPP) to distribute queries and data across all loads
- In the case of an empty table, when importing, Redshift will sample data to create a schema.



DynamoDB CheatSheet

- **DynamoDB** is a fully managed **NoSQL** key/value and document database.
- Applications that contain large amounts of data but require predictable read and write performance while scaling is a good fit for DynamoDB
- DynamoDB scales with whatever **read and write capacity you specify** per second.
- DynamoDB can be set to have **Eventually Consistent Reads (default)** and **Strongly Consistent Reads**
- **Eventually consistent reads** data is returned immediately, but data can be inconsistent. Copies of data will be generally consistent in 1 second.
- **Strongly Consistent Reads** will wait until data is inconsistent. Data will never be inconsistent, but latency will be higher. Copies of data will be consistent with a guarantee of 1 second.
- DynamoDB stores 3 copies of data on SSD drives across 3 regions.



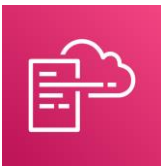
Cognito CheatSheet

- Cognito is a decentralized managed authentication system. When you need to add authentication to your mobile and desktop app easily, *think* Cognito
- **User Pools** user directory allows users to authenticate using OAuth to IdP, such as Facebook, Google, and Amazon to connect to web applications. Cognito User Pool is in itself an IdP
- User Pools use **JWTs** to persist authentication
- **Identity Pools** provide **temporary AWS credentials** to access services eg. S3, DynamoDB
- **Cognito Sync** can sync **user data** and **preferences** across devices with one line of code (powered by SNS)
- **Web Identity Federation** exchange identity and security information between an identity provider (IdP) and an application
- **Identity Provider (IdP)** a trusted provider of your user identity that lets you use authenticate to access other services. eg. Facebook, Twitter, Google, Amazon
- **OIDC** is a type of Identity Provider that uses OAuth
- **SAML** is a type of Identity Provider that is used for Single Sign-On



AWS CLI & SDK CheatSheet

- **CLI** stands for Command Line Interface
- **SDK** stands for Software Development Kit
- The **AWS CLI** lets you interact with AWS from anywhere by simply using a command line
- The **AWS SDK** is a set of API libraries that let you integrate AWS services into your applications.
- **Programmatic Access** must be enabled per user via the IAM console to use CLI or SDK
- **aws configure** command used to set up your AWS credentials for the CLI
- The CLI is installed via a Python script
- Credentials get stored in a plain text file (whenever possible, use roles instead of AWS credentials)
- The SDK is available for the following programming languages
 - C++
 - Go
 - Java
 - JavaScript
 - .NET
 - Node.js
 - PHP
 - Python
 - Ruby



CloudFormation Cheat Sheet

- When being asked to **automate** the provisioning of resources, *think* CloudFormation
- When Infrastructure as Code (IaC) is mentioned, *think* CloudFormation
- CloudFormation can be written in either JSON or YAML
- When CloudFormation encounters an error, it will rollback with **ROLLBACK_IN_PROGRESS**
- CloudFormation templates larger than 51,200 bytes (0.05 MB) are too large to upload directly and must be imported into CloudFormation via an S3 bucket.
- **NestedStacks** helps you break up your CloudFormation template into smaller reusable templates that can be composed into larger templates
- **At least one resource** under resources: must be defined for a CloudFormation template **to be valid**
- CloudFormation **Template Sections**
 - **MetaData** extra information about your template
 - **Description** a description of what the template is supposed to do
 - **Parameters** is how you get user inputs into templates
 - **Transforms** Applies macros (like applying a mod that changes the anatomy to be custom)
 - **Outputs** are values you can use to import into other stacks
 - **Mappings** map keys to values, just like a lookup table
 - **Resources** define the resources you want to provision, **at least one resource is required**
 - **Conditions** are whether resources are created or properties are assigned



Elastic Beanstalk CheatSheet

- **Elastic Beanstalk** handles the deployment, from capacity provisioning, load balancing, and auto-scaling to application health monitoring
- When you want to run a web application, but you don't want to have to think about the underlying infrastructure.
- It costs nothing to use Elastic Beanstalk (only the resources it provisions eg. RDS, ELB, EC2)
- Recommended for test or development apps. Not recommended for production use
- You can choose from the following pre-configured platforms: Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker
- You can run dockerized environments on Elastic Beanstalk.



Storage Gateway CheatSheet

- **Storage Gateway** connects on-premise storage to cloud storage (hybrid storage solution)
- There are four types of Gateways: File Gateway, Volume Gateway, Tape Gateway, and Amazon FSx File Gateway.
- **File Gateway**
 - Lets S3 act as a local file system using NFS or SMB, extends your local hard drive to S3.
 - Supports creating multiple file shares for a single S3 bucket.
 - Local cache size increased up to 64 TiB, providing low-latency access to larger working datasets.
 - Supports S3 Standard, S3 Intelligent-Tiering, S3 Standard-IA, and S3 One Zone-IA.
 - Use cases: migrating on-premises file data to Amazon S3, backing up on-premises file data as objects in Amazon S3, hybrid cloud workflows.
- **Amazon FSx File Gateway**
 - Provides low-latency, on-premises access to fully managed file shares for team file shares, home directories, and file-based business applications.
- **Tape Gateway**
 - Backs up virtual tapes to S3 Glacier for long archive storage.
 - Supports seamless archiving to S3 Glacier and S3 Glacier Deep Archive for cost savings.



Storage Gateway CheatSheet

- **Volume Gateway**

- Used for backups and has two types: Stored and Cached.
- Stored Volume Gateway continuously backs up local storage to S3 as EBS Snapshots. Primary Data on-premise.
- Cached Volume Gateway caches frequently used blocks on-premise. Primary Data is stored on S3.
- Integrates with AWS Backup for centralized and automated backup management.
- Stored Volumes: 1GB to 16TB.
- Cached Volumes: 1GB to 32GB.

- **Additional Features and Improvements:**

- Encryption: Supports AES-128 or AES-256 encryption.
- Log Management: Enhanced system log management to handle low disk space scenarios.
- CloudWatch Integration: File Gateway can be configured with Amazon CloudWatch Log Groups for health and error notifications.



EC2 Pricing – CheatSheet

- EC2 has for 4 pricing models **On-Demand**, **Spot**, **Reserved Instances (RI)**, and **Dedicated**
- **On-Demand** (least commitment)
 - low cost and flexible
 - only pay per hour
 - **Use case:** short-term, spiky, unpredictable workloads, first-time apps
 - Ideal when your workload cannot be interrupted
- **Reserved Instances** up to 75% off (Best long-term value)
 - **Use case:** steady state or predictable usage
 - Can resell unused reserved instances (Reserved Instance Marketplace)
 - Reduced Pricing is based on **Term** x **Class Offering** x **Payment Option**
 - **Payment Terms:** 1-year or 3-year
 - **Payment Options:** All Upfront, Partial Upfront, and No Upfront
 - **Class Offerings**
 - **Standard** Up to 75% reduced pricing compared to on-demand. Cannot change RI Attributes.
 - **Convertible** Up to 54% reduced pricing compared to on-demand. Allows you to change RI Attributes if greater or equal in value.
 - **Scheduled** You reserve instances for specific time periods eg. once a week for a few hours. Savings vary



EC2 Pricing – CheatSheet

- **Spot Pricing** up to 90% off (Biggest Savings)
 - request spare computing capacity
 - flexible start and end times
 - **Use case:** Can handle interruptions (server randomly stopping and starting)
 - **Use case:** For non-critical background jobs
 - Instances can be terminated by AWS **at anytime**
 - If **AWS terminates** your instance, **you don't get charged** for a partial hour of usage.
 - If **you terminate** your instance, **you will still be charged** for any hour it ran.
- **Dedicated Hosting** (Most Expensive)
 - Dedicated servers
 - Can be on-demand or reserved (up to 70% off)
 - **Use case:** When you need a guarantee of isolated hardware (enterprise requirements)